Priority Queue : ordered array implementation

A priority queue is an abstract data type in which each element is associated with a priority.

Unlike an ordinary queue, where elements are removed according to FIFO (First-In, First-Out) order, a priority queue removes the component with the highest priority first.

A priority is a number associated with each component. Higher the numeric value, could mean higher the priority. Or lower the numeric value could mean higher priority.

In our examples, we will assume that higher the numeric value, higher the priority.

Let us relook at the interface of the priority queue before discussing the implementation using an ordered array.

Interface of the priority queue:

The interface of a priority queue is same as that of queue. Component x would have a priority associated with it.

initialize()	Create an empty queue
deinitialize()	Make the queue empty
enqueue(x)	Insert x We may add in number of ways. This in turn would affect dequeue
dequeue()	Remove the element with the highest priority. This in turn depends how enqueue operation was done
isEmpty()	Check whether the queue is empty
isFull()	Check whether the queue is full, for bounded implementations

1. **Implementation of priority queue: Using an ordered array**

We have discussed the implementation of priority queue using an unordered array. Now, we shall discuss the priority queue using an **ordered array**. The queue structure has an array of nodes. The node structure will remain the same. The interface will also remain the same!

We do not have rear and front. Instead, we have n, which is the count of number of nodes in the priority queue.

```
#define MAXSIZE 10
struct queue
{
        job_t key_[MAXSIZE];
        int n_; // # of elements
};
typedef struct queue queue_t;

struct node
{
        job_t jobx_;
        struct node* next_;
};
typedef struct node node_t;

void init_queue(queue_t *ptr_queue);
void deinit_queue(queue_t *ptr_queue);
void enque(queue_t *ptr_queue, job_t *ptr_job);
job_t deque(queue_t *ptr_queue);
int is_empty(queue_t *ptr_queue);
int is_full(queue_t *ptr_queue);
```

The array is ordered based on priority. Let us discuss each one of these operations.

Initialize:

Make the number of elements in the priority queue zero.

```
void init_queue(queue_t *ptr_queue)
{
        ptr_queue->n_ = 0;
}
```

Deinitialize :

There were no resources allocated during any of these processes dynamically.

Just change n to 0 indicating no elements in the priority queue.

```
void deinit_queue(queue_t *ptr_queue)
{
        ptr_queue->n_ = 0;
}
```

Is full :

Check whether the number of elements is equal to the MAXSIZE of the array of nodes.

```
int is_full(queue_t *ptr_queue)
{
        return ptr_queue->n_ == MAXSIZE;
}
```

Is empty:

Check whether the number of elements in the priority queue is 0.

```
int is_empty(queue_t *ptr_queue)

{

        return ptr_queue->n_ == 0;

}
```

Enque :

We want to arrange the nodes such a way that the nodes with the highest priority are at the end of the array. We also want the nodes with some priority which came earlier to be towards higher indices compared to the ones which came later with the same priority.

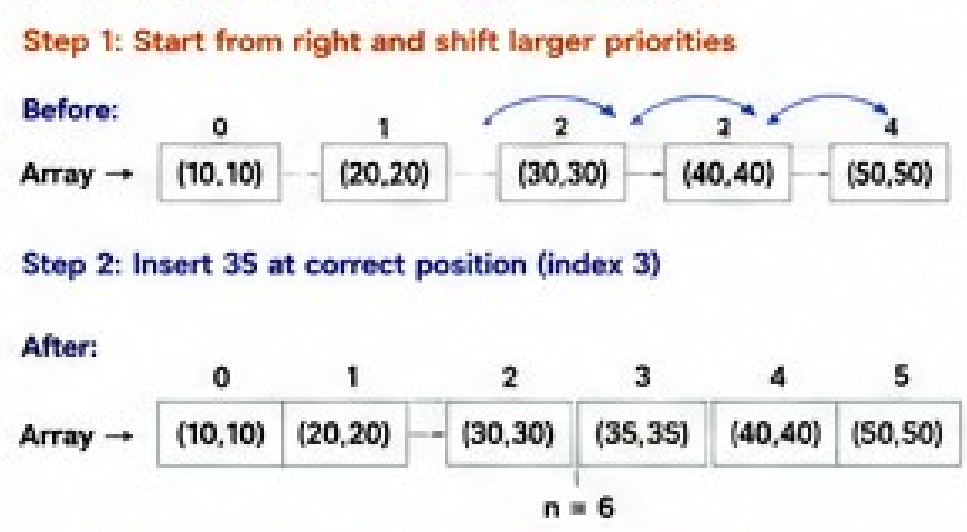
Enque is simple. Remove the last node in the array.

If the array is full, the program is exited.

Otherwise insert the new node at the right position. The nodes are being arranged in non-decreasing order of priority. We basically implement a very famous algorithm called insert. This is used in a sorting algorithm called insertionsort.

We traverse right to left and stop as soon as the right position is found to insert. As we know that we have to insert and make space for a new node, we keep shifting to the right every time we enter the loop.

Once we are out of the loop, we assign the new node to the right position.




```
// insert in order
```

```
void enque(queue_t *ptr_queue, job_t *ptr_job)
{

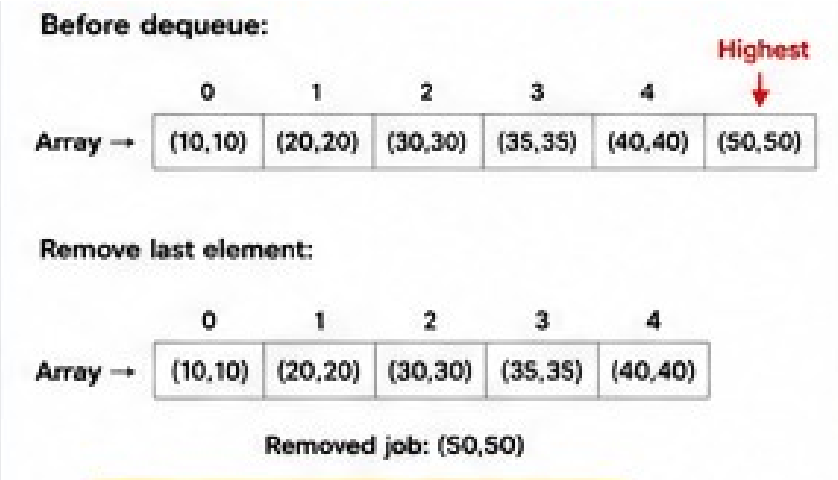
	if(! is_full(ptr_queue))
	{
		int n = ptr_queue->n_;
		int i = n - 1;
		int priority = get_priority(ptr_job);
		while(i >= 0 &&
			get_priority(&ptr_queue->key_[i]) >= priority)
		{
			ptr_queue->key_[i + 1] = ptr_queue->key_[i];
			--i;

		}
		ptr_queue->key_[i + 1] = *ptr_job;
		++ptr_queue->n_;
	}
	else
	{
		printf("queue full; cannot push\n");
		exit(1);
	}
}
```

Deque:

If the array is empty, the program is exited.

We always remove the last node.




```
// always the last one

job_t deque(queue_t *ptr_queue)

{

        if(! is_empty(ptr_queue))

        {

                job_t jobx = ptr_queue->key_[--ptr_queue->n_];

                return jobx;

        }

        else

        {

                printf("queue empty; cannot pop\n");

                exit(1);

        }

}
```

Efficiency of operations :

.

Deque: This requires a few operations. We remove the end as the last node will have the highest priority.

enque: This requires walking through the list from right to left to find the node with the highest priority. While finding the position, the elements shifted to the right. So, the number of operations in the worst case is proportional to the number of nodes in the queue.